VibeLearn LMS — Architecture Document

> **Version:** 1.0
> **Date:** February 2026
> **Based on:** VibeLearn PRD v1.0
> **Status:** Draft for Implementation

Table of Contents

1. System Overview
2. High-Level Architecture
3. Technology Stack
4. Service Topology (Docker)
5. Application Structure
6. Database Architecture
7. Authentication & Authorization
8. API Architecture
9. Content Engine
10. AI Integration Layer
11. Payment System (CloudPayments)
12. Telegram Bot Service
13. File Storage & Media Pipeline
14. Graph Engine
15. Gamification Engine
16. Caching Strategy
17. Real-time Layer
18. Security Architecture

19. Deployment & Infrastructure
20. Observability
21. Performance Budget
22. Disaster Recovery
23. ADR Log

---

1. System Overview

VibeLearn — self-hosted LMS-платформа для обучения вайб-кодингу и применению AI. Архитектура оптимизирована под деплой на одном VPS через Dokploy с возможностью горизонтального масштабирования.

Design Principles

- **Monolith First** — единое Next.js-приложение, без микросервисов на старте. Telegram-бот — единственный отдельный сервис.
- **Database as Source of Truth** — PostgreSQL хранит всё состояние. Redis — только кэш и ephemeral данные.
- **Server-First Rendering** — RSC (React Server Components) по умолчанию, клиентские компоненты только где необходим интерактив.
- **Progressive Enhancement** — контент доступен без JS, интерактив (песочницы, квизы) требует JS.
- **Convention over Configuration** — стандартные паттерны Next.js App Router, Prisma, shadcn/ui.

System Context

```
┌──────────────┐      ┌──────────────┐      ┌──────────────────┐
│   Browser    │──────►│  Traefik     │──────►│  Next.js App     │
│  (Student/   │      │  (Reverse    │      │  (SSR + API)     │
│   Admin)     │◄──────│   Proxy)     │◄──────│                  │
└──────────────┘      └──────────────┘      └──────────┬───────┘
                                                        │
┌──────────────┐                            ┌──────────┴──────┐
│  Telegram    │──────► Telegram Bot ──────►│   PostgreSQL    │
│  (Instructor)│      (Node.js service)     │   + Redis       │
└──────────────┘                            │   + MinIO       │
```

```
                    └──────────┬──────────┘
                               │
                    ┌──────────┴──────────┐
                    │  External Services  │
                    │ • OpenRouter (AI API)
                    │ • CloudPayments     │
                    │ • SMTP (email)      │
                    └─────────────────────┘
```

---

2. High-Level Architecture

```
                    ┌──────────────────────────────────────┐
                    │          VPS (Ubuntu 24.04)          │
                    │          Managed by Dokploy          │
                    │                                      │
│  ┌──────────────────────────────────────────────────────┐  │
│  │                   Docker Network                     │  │
│  │                                                      │  │
│  │  ┌──────────┐   ┌──────────────┐   ┌──────────────────┐  │  │
│  │  │ Traefik  │   │ Next.js App  │   │ Telegram Bot     │  │  │
│  │  │ :80/:443 │──►│ :3000        │   │ :3001            │  │  │
│  │  │ (proxy)  │   │              │   │ (grammy)         │  │  │
│  │  └──────────┘   └──────┬───────┘   └────────┬─────────┘  │  │
│  │                        │                    │            │  │
│  │          ┌─────────────┴────────────────────┘            │  │
│  │          │             │                                 │  │
│  │  ┌───────┴──────┐   ┌──┴──────────┐   ┌──────────────────┐  │  │
│  │  │ PostgreSQL   │   │   Redis     │   │      MinIO       │  │  │
│  │  │ :5432        │   │   :6379     │   │ :9000 / :9001    │  │  │
│  │  │ (primary)    │   │  (Valkey)   │   │ (S3-compatible)  │  │  │
│  │  └──────────────┘   └─────────────┘   └──────────────────┘  │  │
│  │                                                      │  │
│  └──────────────────────────────────────────────────────┘  │
│                                                            │
└────────────────────────────────────────────────────────────┘
```

Request Flow

1. **Client** → HTTPS → **Traefik** (TLS termination, Let's Encrypt)
2. **Traefik** → HTTP → **Next.js** (port 3000)
3. **Next.js** handles:

- SSR/RSC rendering (pages)
- API Route Handlers (`/api/*`)
- Static assets (optimized by Next.js)

4. **Telegram Bot** — отдельный long-polling/webhook сервис, общается напрямую с PostgreSQL и Redis

---

3. Technology Stack

Core

Layer	Technology	Version	Purpose
Runtime	Node.js	20 LTS	Server runtime
Framework	Next.js	15.x	App Router, SSR, API
Language	TypeScript	5.x	Type safety
UI Components	shadcn/ui	latest	Accessible component primitives
Styling	Tailwind CSS	4.x	Utility-first CSS
Database	PostgreSQL	16	Primary data store
ORM	Prisma	6.x	Schema, migrations, typed queries
Cache	Valkey (Redis)	8.x	Sessions, cache, rate limiting
Object Storage	MinIO	latest	Files, media, uploads
Auth	NextAuth.js	5.x (Auth.js)	OAuth, Magic Link, sessions

Content & Editor

Component	Technology	Purpose
Block Editor	Tiptap 2 + @tiptap/pm	WYSIWYG for articles & pages
Code Editor	Monaco Editor	Sandbox code editing
Code Highlighting	Shiki	Server-side syntax highlighting

Graph Editor	React Flow	Visual node-edge editor
Video Player	Plyr / video.js	HLS/DASH playback
DnD	dnd-kit	Drag & drop for course structure

Infrastructure

Component	Technology	Purpose
Orchestration	Dokploy	Docker management, CI/CD
Reverse Proxy	Traefik v3	SSL, routing, load balancing
Telegram Bot	grammY	Telegram Bot API framework
Email	Resend / Nodemailer	Transactional emails
Monitoring	Sentry	Error tracking
Logging	Pino	Structured JSON logging

External APIs

Service	Purpose	Auth Method
OpenRouter	AI model gateway	Bearer token
CloudPayments	Payment processing	Public ID + HMAC
Telegram Bot API	Bot communication	Bot token

---

4. Service Topology

Docker Compose Services

```
# docker-compose.yml (simplified schema)
services:
  app:                    # Next.js application
    build: ./
    ports: ["3000:3000"]
```

```
    depends_on: [postgres, redis, minio]
    environment:
      DATABASE_URL: postgresql://...
      REDIS_URL: redis://redis:6379
      MINIO_ENDPOINT: minio:9000
      OPENROUTER_API_KEY: ${OPENROUTER_API_KEY}
      CLOUDPAYMENTS_PUBLIC_ID: ${CP_PUBLIC_ID}
      CLOUDPAYMENTS_API_SECRET: ${CP_API_SECRET}
      NEXTAUTH_SECRET: ${NEXTAUTH_SECRET}
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/api/health"]
      interval: 30s

  telegram-bot:           # Telegram bot service
    build: ./bot
    depends_on: [postgres, redis]
    environment:
      TELEGRAM_BOT_TOKEN: ${TELEGRAM_BOT_TOKEN}
      DATABASE_URL: postgresql://...
      REDIS_URL: redis://redis:6379
      APP_BASE_URL: https://vibelearn.ru

  postgres:
    image: postgres:16-alpine
    volumes: ["pgdata:/var/lib/postgresql/data"]
    environment:
      POSTGRES_DB: vibelearn
      POSTGRES_USER: vibelearn
      POSTGRES_PASSWORD: ${PG_PASSWORD}
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U vibelearn"]

  redis:
    image: valkey/valkey:8-alpine
    volumes: ["redisdata:/data"]
    command: valkey-server --save 60 1 --loglevel warning
    healthcheck:
      test: ["CMD", "valkey-cli", "ping"]

  minio:
    image: minio/minio:latest
    volumes: ["miniodata:/data"]
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: ${MINIO_ROOT_USER}
      MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
```

```
volumes:
  pgdata:
  redisdata:
  miniodata:
```

Resource Allocation

Service	CPU (cores)	RAM	Disk
Next.js App	1.5	1 GB	—
Telegram Bot	0.25	128 MB	—
PostgreSQL	1.0	1 GB	20 GB
Redis	0.25	256 MB	1 GB
MinIO	0.5	512 MB	40 GB
Traefik	0.25	128 MB	—
Total	**3.75**	**3 GB**	**61 GB**
Recommended VPS	**4 vCPU**	**8 GB**	**80 GB NVMe**

---

5. Application Structure

Directory Layout

```
vibelearn/
├── prisma/
│   ├── schema.prisma          # Database schema
│   ├── migrations/            # Migration history
│   └── seed.ts                # Seed data
│
├── src/
│   ├── app/                   # Next.js App Router
│   │   ├── (public)/          # Public routes group
│   │   │   ├── page.tsx       # Landing page
│   │   │   ├── courses/       # Course catalog & detail
│   │   │   ├── auth/          # Sign in/up pages
│   │   │   └── [slug]/        # Dynamic pages (CMS)
│   │   │
```

```
│   │   ├── (student)/         # Authenticated student routes
│   │   │   ├── layout.tsx     # Student shell (sidebar, header)
│   │   │   ├── dashboard/     # Student dashboard
│   │   │   ├── learn/         # Lesson viewer
│   │   │   │   └── [courseSlug]/[lessonId]/
│   │   │   └── profile/       # Profile & settings
│   │   │
│   │   ├── (admin)/           # Admin routes
│   │   │   ├── layout.tsx     # Admin shell
│   │   │   ├── admin/
│   │   │   │   ├── dashboard/ # Analytics dashboard
│   │   │   │   ├── courses/   # Course CRUD
│   │   │   │   ├── lessons/   # Lesson editors
│   │   │   │   ├── graph/     # Relationship graph editor
│   │   │   │   ├── pages/     # Page editor
│   │   │   │   ├── users/     # User management
│   │   │   │   └── settings/  # AI settings, payments, etc.
│   │   │
│   │   └── api/               # API Route Handlers
│   │       ├── auth/[...nextauth]/
│   │       ├── courses/
│   │       ├── progress/
│   │       ├── ai/
│   │       ├── payments/
│   │       ├── admin/
│   │       ├── upload/
│   │       ├── telegram/webhook/   # Telegram webhook endpoint
│   │       └── health/
│   │
│   ├── components/
│   │   ├── ui/                # shadcn/ui primitives
│   │   ├── shared/            # Shared components
│   │   ├── lesson/            # Lesson renderers by type
│   │   │   ├── ArticleRenderer.tsx
│   │   │   ├── VideoRenderer.tsx
│   │   │   ├── SandboxRenderer.tsx
│   │   │   ├── QuizRenderer.tsx
│   │   │   └── InteractiveRenderer.tsx
│   │   ├── editor/            # Admin editors
│   │   │   ├── BlockEditor.tsx       # Tiptap wrapper
│   │   │   ├── QuizBuilder.tsx       # Quiz constructor
│   │   │   ├── SandboxEditor.tsx     # Sandbox config editor
│   │   │   └── GraphEditor.tsx       # React Flow wrapper
│   │   └── admin/             # Admin-specific components
│   │
│   ├── lib/
│   │   ├── db.ts              # Prisma client singleton
```

```
│   │   ├── redis.ts           # Redis client
│   │   ├── minio.ts           # MinIO client
│   │   ├── auth.ts            # NextAuth config
│   │   ├── openrouter.ts      # OpenRouter API client
│   │   ├── cloudpayments.ts   # CloudPayments helpers
│   │   ├── gamification.ts    # XP, levels, achievements logic
│   │   ├── graph.ts           # Graph validation & queries
│   │   └── rate-limit.ts      # Redis-based rate limiter
│   │
│   ├── services/              # Business logic layer
│   │   ├── course.service.ts
│   │   ├── lesson.service.ts
│   │   ├── progress.service.ts
│   │   ├── payment.service.ts
│   │   ├── ai.service.ts
│   │   ├── gamification.service.ts
│   │   ├── graph.service.ts
│   │   └── user.service.ts
│   │
│   ├── hooks/                 # Client-side React hooks
│   ├── types/                 # Shared TypeScript types
│   └── config/                # App configuration
│       ├── constants.ts
│       ├── levels.ts          # Level definitions
│       └── achievements.ts    # Achievement rules
│
├── bot/                       # Telegram Bot (separate service)
│   ├── src/
│   │   ├── index.ts           # Entry point
│   │   ├── bot.ts             # grammY bot instance
│   │   ├── commands/          # Command handlers
│   │   │   ├── start.ts
│   │   │   ├── note.ts        # Quick note capture
│   │   │   ├── idea.ts        # Lesson/course idea
│   │   │   ├── todo.ts        # Admin todo
│   │   │   ├── stats.ts       # Quick stats
│   │   │   └── announce.ts    # Push announcement
│   │   ├── conversations/     # Multi-step conversations
│   │   │   ├── create-lesson.ts
│   │   │   └── quick-quiz.ts
│   │   ├── middleware/        # Auth, logging
│   │   ├── services/          # Shared DB access
│   │   └── keyboard/          # Inline keyboards
│   ├── Dockerfile
│   ├── package.json
│   └── tsconfig.json
│
```

```
├── docker-compose.yml
├── Dockerfile
├── .env.example
└── architecture.md            # This document
```

Layer Architecture

```
┌──────────────────────────────────────────────┐
│                  Presentation                 │
│  Next.js Pages (RSC) │ API Routes │ Bot CLI   │
├──────────────────────────────────────────────┤
│                  Application                  │
│  Services (business logic, validation)        │
├──────────────────────────────────────────────┤
│                Infrastructure                 │
│  Prisma │ Redis │ MinIO │ OpenRouter │ SMTP   │
├──────────────────────────────────────────────┤
│                   Storage                     │
│  PostgreSQL │ Valkey │ MinIO S3 │ Filesystem  │
└──────────────────────────────────────────────┘
```

Правило: API Routes и Bot вызывают только Services. Services вызывают Infrastructure (lib/). Никогда не обращаемся к БД напрямую из компонентов или роутов — только через сервисный слой.

---

6. Database Architecture

ER Diagram (Core)

```
┌──────────────┐      ┌──────────────┐      ┌──────────────┐
│    users     │      │   courses    │      │    pages     │
├──────────────┤      ├──────────────┤      ├──────────────┤
│ id (PK)      │      │ id (PK)      │      │ id (PK)      │
│ email        │      │ slug         │      │ slug         │
│ name         │      │ title        │      │ title        │
│ avatar_url   │      │ description  │      │ blocks       │
│ role         │      │ cover_image  │      │ seo_meta     │
│ xp           │      │ price_rub    │      │ status       │
│ level        │      │ status       │      │ created_at   │
│ streak_days  │      │ difficulty   │      │ updated_at   │
│ streak_last  │      │ metadata     │      └──────────────┘
│ preferences  │      │ created_at   │
```

```
| telegram_id  |      | updated_at   |
| created_at   |      └──────┬───────┘
└──────┬───────┘             │
       │              ┌──────┴──────┐
       │              │  modules    │
       │              ├─────────────┤
       │              | id (PK)     |
       │              | course_id   |──→ courses.id
       │              | title       |
       │              | order_index|
       │              | unlock_cond|
       │              └──────┬──────┘
       │                     │
       │              ┌──────┴──────┐
       │              │  lessons    │
       │              ├─────────────┤
       │              | id (PK)     |
       │              | module_id   |──→ modules.id
       │              | type        |  (article|video|sandbox|quiz|interactive|med
       │              | title       |
       │              | content     |  (JSONB — structure varies by type)
       │              | order_index|
       │              | xp_reward   |
       │              | est_minutes|
       │              | is_preview  |
       │              └─────────────┘
       │
       │
  ┌────┴──────────┐   ┌──────────────┐   ┌──────────────────┐
  | enrollments   |   |  progress    |   |  content_graph   |
  ├───────────────┤   ├──────────────┤   ├──────────────────┤
  | id (PK)       |   | id (PK)      |   | id (PK)          |
  | user_id       |──→| user_id      |   | source_type      |
  | course_id     |   | lesson_id    |   | source_id        |
  | payment_id    |   | status       |   | target_type      |
  | enrolled_at   |   | score        |   | target_id        |
  └───────────────┘   | attempts     |   | relation_type    |
                      | answers_data |   | metadata         |
  ┌──────────────┐    | xp_earned    |   └──────────────────┘
  |  payments    |    | started_at   |
  ├──────────────┤    | completed_at |   ┌──────────────────┐
  | id (PK)      |    └──────────────┘   | achievements     |
  | user_id      |                       ├──────────────────┤
  | course_id    |   ┌──────────────┐    | id (PK)          |
  | transaction_id|  |  comments    |    | slug             |
  | amount_kop   |   ├──────────────┤    | title            |
  | status       |   | id (PK)      |    | description      |
  | provider_data|   | user_id      |    | icon_url         |
```

```
│ created_at    │    │ lesson_id     │    │ condition (JSONB)│
└───────────────┘    │ parent_id     │    │ xp_bonus         │
                     │ content       │    └──────────────────┘
┌───────────────┐    │ created_at    │
│ai_interactions│    └───────────────┘    ┌──────────────────┐
├───────────────┤                         │user_achievements │
│ id (PK)       │    ┌───────────────┐    ├──────────────────┤
│ user_id       │    │  admin_notes  │    │ user_id          │
│ lesson_id     │    ├───────────────┤    │ achievement_id   │
│ model         │    │ id (PK)       │    │ earned_at        │
│ tokens_in     │    │ author_tg_id  │    └──────────────────┘
│ tokens_out    │    │ type          │
│ cost_usd      │    │ content       │
│ prompt_hash   │    │ related_type  │
│ created_at    │    │ related_id    │
└───────────────┘    │ created_at    │
                     └───────────────┘
```

Prisma Schema (Key Enums)

```
enum UserRole {
  student
  instructor
  admin
}

enum CourseStatus {
  draft
  published
  archived
}

enum LessonType {
  article
  video
  sandbox
  quiz
  interactive
  media
}

enum ProgressStatus {
  not_started
  in_progress
  completed
}
```

```
enum RelationType {
  prerequisite    // A must be completed before B
  recommended     // A is recommended before B
  related         // Bidirectional relation
  unlocks         // Completing A unlocks access to B
}

enum ContentNodeType {
  course
  module
  lesson
  page
}

enum PaymentStatus {
  pending
  completed
  refunded
  failed
}
```

Key Indexes

```
-- Performance-critical indexes
CREATE INDEX idx_progress_user_lesson ON progress(user_id, lesson_id);
CREATE INDEX idx_progress_user_status ON progress(user_id, status);
CREATE INDEX idx_enrollments_user ON enrollments(user_id);
CREATE INDEX idx_enrollments_course ON enrollments(course_id);
CREATE INDEX idx_lessons_module_order ON lessons(module_id, order_index);
CREATE INDEX idx_modules_course_order ON modules(course_id, order_index);
CREATE INDEX idx_content_graph_source ON content_graph(source_type, source_id);
CREATE INDEX idx_content_graph_target ON content_graph(target_type, target_id);
CREATE INDEX idx_comments_lesson ON comments(lesson_id, created_at DESC);
CREATE INDEX idx_ai_interactions_user_day ON ai_interactions(user_id, created_at)
CREATE INDEX idx_payments_transaction ON payments(transaction_id);
CREATE INDEX idx_users_telegram ON users(telegram_id) WHERE telegram_id IS NOT NU
CREATE INDEX idx_admin_notes_related ON admin_notes(related_type, related_id);
```

JSONB Content Schemas

Каждый `LessonType` определяет структуру поля `content`:

```
// article
interface ArticleContent {
```

```
  blocks: TiptapJSONContent; // Tiptap JSON document
}

// video
interface VideoContent {
  video_url: string;
  thumbnail_url?: string;
  duration_sec: number;
  chapters: Array<{ time_sec: number; title: string }>;
  captions_vtt_url?: string;
}

// sandbox
interface SandboxContent {
  sandbox_type: "prompt" | "code";
  task_description: string;         // Markdown
  starter_code?: string;
  reference_solution?: string;
  evaluation_criteria: string;      // Prompt for AI judge
  model_whitelist: string[];        // e.g., ["claude-sonnet-4-5"]
  max_attempts_per_day: number;
  hints: string[];
  // For code sandbox:
  language?: "html" | "css" | "javascript" | "react";
  test_code?: string;               // Automated tests
}

// quiz
interface QuizContent {
  questions: Array<{
    id: string;
    type: "single" | "multiple" | "truefalse" | "freetext" | "dragdrop" | "code_c
    text: string;                   // Markdown
    options?: Array<{ id: string; text: string; is_correct?: boolean }>;
    correct_answer?: string | string[];
    explanation: string;            // Shown after submit
    points: number;
  }>;
  pass_threshold: number;           // 0-100 percentage
  shuffle_questions: boolean;
  shuffle_options: boolean;
  time_limit_sec?: number;
  max_attempts: number;
}

// interactive
interface InteractiveContent {
```

```
  interactive_type: "step_by_step" | "fill_blank" | "before_after" | "hotspot";
  steps?: Array<{ instruction: string; validation: object }>;
  // ... type-specific fields
}

// media
interface MediaContent {
  media_type: "image" | "gif" | "gallery" | "embed" | "pdf";
  url?: string;
  urls?: string[];                  // For gallery
  embed_html?: string;              // For embed
  caption?: string;
}
```

---

7. Authentication & Authorization

Auth Flow

```
┌────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐
│ Client │──►│ NextAuth │──►│ Provider │──►│ Callback │
│        │    │ /api/auth│    │ (Google/ │    │ Create/  │
│        │◄──│          │◄──│  Email)  │◄──│ Link User│
└────────┘    └──────────┘    └──────────┘    └──────────┘
```

Providers

1. **Email Magic Link** — Resend/Nodemailer adapter. Passwordless. Primary method.
2. **Google OAuth** — via `@auth/google`. Для удобства.
3. **Telegram Login** (future) — linking Telegram account to user profile.

Session Strategy

- **JWT sessions** — stateless, хранятся в HTTP-only secure cookie.
- JWT содержит: `userId`, `role`, `email`, `name`.
- Refresh: rolling 30-day session, renewed on activity.

Authorization Model (RBAC)

```
const PERMISSIONS = {
```

```
  student: [
    "courses:read",
    "lessons:read",
    "progress:write",
    "comments:write",
    "profile:write",
    "ai:use",
  ],
  instructor: [
    "...student",
    "courses:write",
    "lessons:write",
    "pages:write",
    "graph:write",
    "comments:moderate",
    "admin:dashboard:read",
    "admin:notes:write",        // Via Telegram bot
  ],
  admin: [
    "...instructor",
    "users:manage",
    "payments:read",
    "settings:write",
    "ai:configure",
  ],
} as const;
```

Middleware Chain

```
// src/middleware.ts
export function middleware(request: NextRequest) {
  // 1. Rate limiting (Redis)
  // 2. Auth check (JWT validation)
  // 3. Role-based route protection:
  //    /admin/*  → admin or instructor
  //    /api/admin/* → admin or instructor
  //    /learn/*  → any authenticated user
  //    /api/ai/* → authenticated + enrolled
  // 4. CSRF validation (for mutations)
}
```

---

8. API Architecture

Route Handler Convention

Все API-эндпоинты следуют единообразной структуре:

```
// src/app/api/[resource]/route.ts
import { auth } from "@/lib/auth";
import { requireRole } from "@/lib/middleware";
import { courseService } from "@/services/course.service";

export async function GET(request: Request) {
  // 1. Auth
  const session = await auth();
  if (!session) return Response.json({ error: "Unauthorized" }, { status: 401 });

  // 2. Validate input (zod)
  const params = searchParamsSchema.parse(/* ... */);

  // 3. Business logic (service layer)
  const result = await courseService.list(params);

  // 4. Response
  return Response.json(result);
}
```

API Map

Public

Method	Path	Description
GET	`/api/courses`	Каталог курсов (published)
GET	`/api/courses/:slug`	Детали курса + модули
GET	`/api/pages/:slug`	CMS-страница
POST	`/api/auth/[...nextauth]`	Auth handlers
GET	`/api/health`	Health check

Student (auth required)

Method	Path	Description

GET	`/api/courses/:slug/lessons/:id`	Контент урока (проверка enrollment)
POST	`/api/progress/:lessonId/complete`	Отметка завершения
POST	`/api/progress/:lessonId/quiz`	Сабмит ответов квиза
POST	`/api/progress/:lessonId/sandbox`	Сабмит решения песочницы
GET	`/api/me/progress`	Общий прогресс
GET	`/api/me/achievements`	Бейджи
PATCH	`/api/me/profile`	Обновление профиля
GET	`/api/comments?lessonId=`	Комментарии к уроку
POST	`/api/comments`	Создание комментария

AI (auth + rate limit)

Method	Path	Description
POST	`/api/ai/completion`	Промпт-плейграунд (streaming SSE)
POST	`/api/ai/evaluate`	AI-оценка песочницы
POST	`/api/ai/hint`	AI-подсказка

Payments

Method	Path	Description
POST	`/api/payments/create-order`	Инициация оплаты
POST	`/api/payments/confirm`	Webhook: успешная оплата
POST	`/api/payments/fail`	Webhook: ошибка
POST	`/api/payments/refund`	Webhook: возврат

Admin (admin/instructor role)

Method	Path	Description

CRUD	`/api/admin/courses/*`	Управление курсами
CRUD	`/api/admin/modules/*`	Управление модулями
CRUD	`/api/admin/lessons/*`	Управление уроками
CRUD	`/api/admin/pages/*`	Управление страницами
CRUD	`/api/admin/graph/*`	Управление связями
CRUD	`/api/admin/achievements/*`	Управление бейджами
GET	`/api/admin/users`	Список пользователей
PATCH	`/api/admin/users/:id`	Обновление пользователя
GET	`/api/admin/analytics/*`	Метрики
PATCH	`/api/admin/settings/*`	Настройки AI, платежей

Telegram

Method	Path	Description
POST	`/api/telegram/webhook`	Telegram webhook endpoint (alternative to polling)

Error Response Format

```
{
  "error": {
    "code": "ENROLLMENT_REQUIRED",
    "message": "You need to purchase this course first",
    "status": 403
  }
}
```

Validation

Все входные данные валидируются через **Zod**:

```
// src/types/api.ts
import { z } from "zod";

export const createCommentSchema = z.object({
```

```
  lessonId: z.string().uuid(),
  content: z.string().min(1).max(5000),
  parentId: z.string().uuid().optional(),
});
```

---

9. Content Engine

Rendering Pipeline

```
┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐
│    Database      │      │   Server (RSC)   │      │     Client       │
│  lesson.content  │──────►│ ContentRenderer  │──────►│  Interactive     │
│  (JSONB)         │      │ (by lesson.type) │      │  components      │
└──────────────────┘      └──────────────────┘      └──────────────────┘
```

Content Renderer Strategy

```
// src/components/lesson/LessonRenderer.tsx (Server Component)
export function LessonRenderer({ lesson }: { lesson: Lesson }) {
  switch (lesson.type) {
    case "article":
      return <ArticleRenderer content={lesson.content} />;
    case "video":
      return <VideoRenderer content={lesson.content} />;
    case "sandbox":
      return <SandboxRenderer content={lesson.content} lessonId={lesson.id} />;
    case "quiz":
      return <QuizRenderer content={lesson.content} lessonId={lesson.id} />;
    case "interactive":
      return <InteractiveRenderer content={lesson.content} lessonId={lesson.id} /
    case "media":
      return <MediaRenderer content={lesson.content} />;
  }
}
```

Block Editor Architecture (Admin)

```
┌──────────────────────────────────────────────┐
│              Tiptap Editor                   │
│                                              │
│  ┌─────────┐  ┌─────────┐  ┌──────────────┐ │
```

```
│  │ Heading │  │  Text   │  │   CodeBlock   │ │
│  │  node   │  │  node   │  │  (Shiki HL)   │ │
│  └─────────┘  └─────────┘  └───────────────┘ │
│  ┌─────────┐  ┌─────────┐  ┌───────────────┐ │
│  │  Image  │  │   GIF   │  │   Callout     │ │
│  │  node   │  │  node   │  │   node        │ │
│  └─────────┘  └─────────┘  └───────────────┘ │
│  ┌─────────┐  ┌─────────┐  ┌───────────────┐ │
│  │  Embed  │  │ Spoiler │  │    Table      │ │
│  │  node   │  │  node   │  │    node       │ │
│  └─────────┘  └─────────┘  └───────────────┘ │
│                                              │
│  Output: TiptapJSONContent → stored in JSONB │
└──────────────────────────────────────────────┘
```

Media Upload Flow

```
Client                    API                     MinIO
  │                        │                        │
  │  POST /api/upload      │                        │
  │  (multipart/form-data) │                        │
  │──────────────────────►│                        │
  │                        │  putObject(bucket,     │
  │                        │    key, buffer)        │
  │                        │──────────────────────►│
  │                        │                        │
  │                        │◄── objectUrl ─────────│
  │                        │                        │
  │◄── { url, key, size } ─│                        │
  │                        │                        │
```

Upload constraints:

- Images: max 10 MB, formats: jpg/png/webp/gif
- Video: max 2 GB, formats: mp4/webm (через presigned URL для больших файлов)
- Bucket structure: `/{type}/{year}/{month}/{uuid}.{ext}`

---

10. AI Integration Layer

Architecture

```
┌─────────┐      ┌──────────────────┐      ┌──────────────┐      ┌────────────┐
│ Client  │──────►│ /api/ai/*        │──────►│  AI Service  │──────►│ OpenRouter │
│ (SSE)   │◄──────│ (Route Handler)  │◄──────│              │◄──────│ API        │
└─────────┘      └──────────────────┘      └──────────────┘      └────────────┘
                        │                         │
                        ▼                         ▼
                  ┌──────────┐            ┌──────────────┐
                  │Rate Limiter│          │ai_interactions│
                  │  (Redis)  │           │  (logging)   │
                  └──────────┘            └──────────────┘
```

AI Service Interface

```
// src/services/ai.service.ts

interface AICompletionRequest {
  lessonId: string;
  userId: string;
  userPrompt: string;
  model?: string;           // Override from lesson config
}

interface AIEvaluationRequest {
  lessonId: string;
  userId: string;
  userOutput: string;       // What the student produced
  referenceSolution: string;
  evaluationCriteria: string;
}

class AIService {
  async completion(req: AICompletionRequest): AsyncGenerator<string>;
  async evaluate(req: AIEvaluationRequest): Promise<{ score: number; feedback: st
  async hint(lessonId: string, userId: string, currentCode: string): Promise<stri
}
```

OpenRouter Integration

```
// src/lib/openrouter.ts

const OPENROUTER_BASE = "https://openrouter.ai/api/v1";

interface OpenRouterRequest {
  model: string;
```

```
  messages: Message[];
  stream?: boolean;
  max_tokens?: number;
  temperature?: number;
}

async function* streamCompletion(req: OpenRouterRequest): AsyncGenerator<string>
  const response = await fetch(`${OPENROUTER_BASE}/chat/completions`, {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${process.env.OPENROUTER_API_KEY}`,
      "Content-Type": "application/json",
      "HTTP-Referer": process.env.APP_URL,
      "X-Title": "VibeLearn LMS",
    },
    body: JSON.stringify({ ...req, stream: true }),
  });

  // Parse SSE stream, yield tokens
  // Log usage to ai_interactions table
}
```

Rate Limiting Strategy

```
Per-user limits (configurable in admin):
├── Global: 100 AI requests / day
├── Per-lesson: 20 requests / day
└── Concurrent: 2 simultaneous requests

Per-system limits:
├── Monthly budget: $500 (configurable)
├── Alert at 80%: email to admin
└── Hard stop at 100%: AI features disabled
```

Model Selection

Scenario	Default Model	Fallback	Max Tokens
Prompt Playground	claude-sonnet-4-5	gpt-4o	4096
AI Evaluation	claude-sonnet-4-5	gpt-4o	2048
Hints	claude-haiku-4-5	gpt-4o-mini	1024
Free-text quiz	claude-haiku-4-5	gpt-4o-mini	512

11. Payment System

CloudPayments Integration Flow

```
┌──────────┐      ┌──────────────┐      ┌──────────────────┐      ┌──────────────────┐
│  Client  │      │  CP Widget   │      │  CloudPayments   │      │  Our Server      │
│          │      │  (iframe)    │      │  Server          │      │  (webhooks)      │
│          │      │              │      │                  │      │                  │
│ Click ──►│──────►│ Card form    │      │                  │      │                  │
│ "Buy"    │      │ + 3DS        │──────►│ Process payment  │      │                  │
│          │      │              │      │                  │──────►│ POST /confirm    │
│          │      │              │      │                  │      │ Verify HMAC      │
│          │      │              │      │                  │      │ Create enroll    │
│          │      │              │      │                  │◄──────│ {code: 0}        │
│          │◄──────│ Success      │◄──────│ Return result    │      │                  │
│          │      │ callback     │      │                  │      │                  │
└──────────┘      └──────────────┘      └──────────────────┘      └──────────────────┘
```

Webhook Verification

```
// src/lib/cloudpayments.ts

function verifyWebhook(body: string, hmacHeader: string): boolean {
  const expectedHmac = crypto
    .createHmac("sha256", process.env.CLOUDPAYMENTS_API_SECRET!)
    .update(body)
    .digest("base64");
  return crypto.timingSafeEqual(
    Buffer.from(hmacHeader),
    Buffer.from(expectedHmac)
  );
}
```

Payment State Machine

```
            ┌──────────┐
            │ pending  │
            └──────┬───┘
                   │
      ┌────────────┼────────────┐
```

```
    ▼          ▼          ▼
┌─────────┐  ┌────────┐  ┌─────────┐
│completed│  │failed  │  │cancelled│
└─────────┘  └────────┘  └─────────┘
     │
     ▼
┌────────┐
│refunded│
└────────┘
```

Idempotency

Каждый webhook содержит `TransactionId`. Перед обработкой проверяем, существует ли уже запись с таким `transaction_id` в таблице `payments`. Если да — возвращаем `{code: 0}` без повторной обработки.

---

12. Telegram Bot Service

Purpose

Telegram-бот — инструмент для инструктора/admina для быстрой фиксации информации без входа в веб-admинку. **Не** является заменой веб-интерфейса, а его дополнением для мобильного workflow.

Architecture

```
┌──────────────────┐      ┌──────────────────────────────┐      ┌──────────────────┐
│   Telegram       │      │    Bot Service               │      │  PostgreSQL      │
│   Messenger      │──────►│    (Node.js + grammY)        │──────►│  (shared DB)     │
│                  │◄──────│                              │◄──────│                  │
└──────────────────┘      │  ┌────────────────────────┐  │      └──────────────────┘
                          │  │ Command Router         │  │
                          │  │ • /note                │  │      ┌──────────────────┐
                          │  │ • /idea                │  │──────►│    Redis         │
                          │  │ • /todo                │  │      │  (shared)        │
                          │  │ • /stats               │  │      └──────────────────┘
                          │  │ • /announce            │  │
                          │  │ • /quiz                │  │      ┌──────────────────┐
                          │  │ • /link                │  │──────►│    MinIO         │
                          │  └────────────────────────┘  │      │  (shared)        │
                          └──────────────────────────────┘      └──────────────────┘
```

Deployment Mode

Два варианта (настраивается в env):

1. **Long Polling** (default, проще для dev) — бот сам опрашивает Telegram API
2. **Webhook** — Telegram шлёт запросы на `https://vibelearn.ru/api/telegram/webhook`, Next.js проксирует в бот-сервис

Commands

Command	Description	Role
`/start`	Привязка Telegram к аккаунту VibeLearn. Генерирует одноразовый код для ввода в веб-интерфейсе	any
`/note <text>`	Быстрая заметка — сохраняется в `admin_notes` с типом `note`	instructor+
`/idea [course|lesson] <text>`	Идея для курса/урока — сохраняется с типом `idea`	instructor+
`/todo <text>`	Задача — сохраняется с типом `todo`, показывается в админ-дашборде	instructor+
`/stats`	Быстрая статистика: активные ученики сегодня, новые регистрации, доход за неделю	instructor+
`/announce <text>`	Отправить уведомление всем активным ученикам (через email + in-app)	admin
`/quiz`	Интерактивное создание квиз-вопроса прямо в чате (multi-step conversation)	instructor+
`/link <courseSlug>`	Получить прямую ссылку на курс/урок	instructor+
`/revenue [period]`	Отчёт по доходам за период	admin

Data Model (admin_notes)

```
interface AdminNote {
  id: string;
```

```
  author_telegram_id: bigint;  // Telegram user ID
  type: "note" | "idea" | "todo";
  content: string;
  related_type?: "course" | "lesson" | "module" | "page" | null;
  related_id?: string | null;
  is_done: boolean;           // For todos
  created_at: Date;
}
```

Bot Authentication

```
1. User sends /start to bot
2. Bot generates 6-digit code, stores in Redis (TTL: 5 min)
   Key: telegram:link:{code} → { telegramId, telegramUsername }
3. Bot replies: "Введите код XXXXXX в настройках профиля на vibelearn.ru"
4. User enters code in web profile → /api/me/link-telegram
5. Server verifies code in Redis, adds telegram_id to user record
6. Bot now checks telegram_id against users table for role-based access
```

Bot → Web Sync

Заметки и идеи, созданные через бот, отображаются в админ-панели:

- **Dashboard widget** — последние заметки и идеи
- **Todo list** — в отдельном разделе админки с возможностью отметить выполненным
- **Course/Lesson sidebar** — привязанные заметки показываются рядом с соответствующим контентом

Conversation Flow (Multi-step Example: /quiz)

```
User: /quiz
Bot:  Для какого урока создаём вопрос?
      [Inline keyboard: list of recent lessons]
User: [clicks "Урок 3: Промпт-инжиниринг"]
Bot:  Тип вопроса?
      [single | multiple | truefalse | freetext]
User: [clicks "single"]
Bot:  Напишите вопрос:
User: Какой символ используется для system prompt в ChatGPT?
Bot:  Варианты ответа (по одному в строке, правильный начинается с *):
User: /system
      *Нет специального символа
      #prompt
```

```
      @system
Bot:  ✅ Вопрос создан и добавлен в урок "Промпт-инжиниринг"!
      Добавить объяснение? (можно отправить текстом или /skip)
User: Правильный ответ — нет специального символа...
Bot:  ✅ Объяснение сохранено. Создать ещё вопрос? /quiz
```

Bot Configuration

```
// bot/src/config.ts
export const BOT_CONFIG = {
  // Only these Telegram user IDs can use instructor+ commands
  // Mapped from users.telegram_id where role IN ('instructor', 'admin')
  allowedRoles: ["instructor", "admin"],

  // Rate limits for bot commands
  rateLimit: {
    commands_per_minute: 30,
    announce_cooldown_minutes: 60,
  },

  // Notification settings
  notifications: {
    newEnrollment: true,     // Notify instructor on new enrollment
    dailyDigest: true,       // Daily stats at 9:00 AM
    paymentAlert: true,      // Notify on successful payment
    errorAlert: true,        // Notify on system errors
  },
};
```

---

13. File Storage & Media Pipeline

MinIO Bucket Structure

```
vibelearn-media/
├── courses/              # Course cover images
│   └── {courseId}/
│       └── cover.{ext}
├── lessons/              # Lesson media
│   └── {lessonId}/
│       ├── video.mp4
│       ├── captions.vtt
│       └── media/
```

```
│           └── {uuid}.{ext}
├── uploads/              # Editor uploads (images, GIFs)
│   └── {year}/{month}/
│       └── {uuid}.{ext}
├── avatars/              # User avatars
│   └── {userId}.{ext}
└── exports/              # Generated exports (certificates, CSV)
    └── {date}/
```

Upload Pipeline

```
// Small files (<10MB): Direct upload through API
POST /api/upload
  → validate (auth, file type, size)
  → sharp resize/optimize (images)
  → putObject to MinIO
  → return URL

// Large files (>10MB, e.g., video): Presigned URL
POST /api/upload/presign
  → validate auth
  → generate presigned PUT URL (1h TTL)
  → return { uploadUrl, fileKey }
// Client uploads directly to MinIO
// Client confirms: POST /api/upload/confirm { fileKey }
```

Image Processing

Все загружаемые изображения обрабатываются через **sharp**:

- Resize: max 1920px по длинной стороне
- Format: WebP (с fallback на original для GIF)
- Quality: 80%
- Thumbnails: 400px для превью

---

14. Graph Engine

Data Structure

Граф контента хранится в таблице `content_graph` (edge list) и визуализируется через

React Flow в админке.

Graph Operations

```
// src/services/graph.service.ts

class GraphService {
  // CRUD
  async addEdge(source: NodeRef, target: NodeRef, type: RelationType): Promise<Ed
  async removeEdge(edgeId: string): Promise<void>;
  async getEdges(nodeRef: NodeRef): Promise<Edge[]>;

  // Queries
  async getPrerequisites(nodeRef: NodeRef): Promise<NodeRef[]>;
  async getDependents(nodeRef: NodeRef): Promise<NodeRef[]>;
  async getRelated(nodeRef: NodeRef): Promise<NodeRef[]>;

  // Graph for React Flow
  async getFullGraph(filter?: GraphFilter): Promise<{ nodes: RFNode[]; edges: RFE
  async getSubgraph(rootNodeRef: NodeRef, depth: number): Promise<{ nodes: RFNode

  // Validation
  async validateNoCycles(source: NodeRef, target: NodeRef): Promise<boolean>;
  async findOrphans(): Promise<NodeRef[]>;
  async findUnreachable(fromCourse: string): Promise<NodeRef[]>;
}
```

Cycle Detection

При добавлении `prerequisite` или `unlocks` связи выполняется BFS/DFS от target к source. Если source достижим — связь отклоняется с ошибкой.

```
async validateNoCycles(source: NodeRef, target: NodeRef): Promise<boolean> {
  // BFS from target following prerequisite/unlocks edges
  // If we reach source — cycle detected, return false
  const visited = new Set<string>();
  const queue = [target];

  while (queue.length > 0) {
    const current = queue.shift()!;
    const key = `${current.type}:${current.id}`;
    if (key === `${source.type}:${source.id}`) return false; // Cycle!
    if (visited.has(key)) continue;
    visited.add(key);
```

```
    const edges = await this.getPrerequisites(current);
    queue.push(...edges);
  }
  return true; // No cycle
}
```

React Flow Integration

```
// Node types for React Flow
const NODE_STYLES = {
  course:  { color: "#6C5CE7", icon: BookOpen },
  module:  { color: "#00B894", icon: Folder },
  lesson:  { color: "#FDCB6E", icon: FileText },
  page:    { color: "#B2BEC3", icon: Globe },
};

// Edge styles
const EDGE_STYLES = {
  prerequisite: { style: "solid",  color: "#E17055", animated: false, label: "req
  recommended:  { style: "dashed", color: "#74B9FF", animated: false, label: "rec
  related:      { style: "dotted", color: "#A29BFE", animated: false, label: "rel
  unlocks:      { style: "solid",  color: "#00B894", animated: true,  label: "unl
};
```

---

15. Gamification Engine

Architecture

```
// src/services/gamification.service.ts

class GamificationService {
  // Triggered after lesson completion, quiz submit, etc.
  async processEvent(event: GamificationEvent): Promise<GamificationResult> {
    const xpGained = this.calculateXP(event);
    const newTotal = await this.addXP(event.userId, xpGained);
    const levelUp = this.checkLevelUp(newTotal);
    const streak = await this.updateStreak(event.userId);
    const newAchievements = await this.checkAchievements(event.userId, event);

    return { xpGained, newTotal, levelUp, streak, newAchievements };
  }
}
```

```
type GamificationEvent =
  | { type: "lesson_completed"; userId: string; lessonId: string; score?: number
  | { type: "quiz_perfect"; userId: string; lessonId: string }
  | { type: "sandbox_high_score"; userId: string; lessonId: string; score: number
  | { type: "daily_login"; userId: string }
  | { type: "comment_posted"; userId: string; commentId: string };
```

Level Calculation

```
// src/config/levels.ts
export const LEVELS = [
  { level: 1, xp: 0,    title: "Новичок" },
  { level: 2, xp: 100,  title: "Промпт-падаван" },
  { level: 3, xp: 300,  title: "AI-практик" },
  { level: 4, xp: 600,  title: "Вайб-кодер" },
  { level: 5, xp: 1000, title: "Вайб-архитектор" },
  { level: 6, xp: 1500, title: "AI-мастер" },
  { level: 7, xp: 2500, title: "Вайб-сенсей" },
];

function getLevelForXP(xp: number): Level {
  return LEVELS.findLast(l => xp >= l.xp)!;
}
```

Achievement Rules

Achievements определяются как JSON-правила в базе, проверяемые при каждом событии:

```
// Achievement condition examples (stored in JSONB)
{ "type": "progress_count", "filter": { "status": "completed", "lesson_type": "sa
{ "type": "quiz_perfect_count", "threshold": 5 }
{ "type": "streak_days", "threshold": 7 }
{ "type": "total_xp", "threshold": 1000 }
{ "type": "course_completed", "course_id": "any" }
```

---

16. Caching Strategy

Redis Key Namespace

```
vibelearn:                          # App prefix
├── session:{token}                 # JWT session data (if needed)
├── rate:{userId}:{endpoint}:{date} # Rate limiting counters
├── cache:
│   ├── course:{slug}               # Course detail (TTL: 5min)
│   ├── course:list                 # Course catalog (TTL: 1min)
│   ├── lesson:{id}:content         # Lesson content (TTL: 5min)
│   ├── user:{id}:progress          # User progress summary (TTL: 30s)
│   └── graph:full                  # Full graph for React Flow (TTL: 1min)
├── ai:
│   ├── eval:{promptHash}           # Cached AI evaluation (TTL: 1h)
│   ├── budget:{userId}:{date}      # Daily AI spend per user
│   └── budget:global:{month}       # Monthly global AI spend
├── streak:{userId}                 # Streak data (TTL: 48h)
└── telegram:
    ├── link:{code}                 # Telegram linking code (TTL: 5min)
    └── conv:{telegramId}           # Conversation state (TTL: 30min)
```

Cache Invalidation

- **Course/Lesson changes** → delete `cache:course:*` and `cache:lesson:{id}:*`
- **Graph changes** → delete `cache:graph:*`
- **Progress updates** → delete `cache:user:{id}:progress`
- **TTL-based expiry** as fallback for all keys

Pattern: Read-through cache

```
async function cached<T>(key: string, ttlSec: number, fn: () => Promise<T>): Prom
  const cached = await redis.get(key);
  if (cached) return JSON.parse(cached);

  const result = await fn();
  await redis.set(key, JSON.stringify(result), "EX", ttlSec);
  return result;
}
```

---

17. Real-time Layer

Implementation: Server-Sent Events (SSE)

Для MVP используем SSE вместо WebSocket — проще, достаточно для наших кейсов:

Feature	Transport	Description
AI streaming	SSE	Streaming response tokens
New comments	SSE (polling fallback)	Уведомление о новых комментариях
XP/Level updates	SSE	Toast при получении XP/бейджа
Notifications	SSE	In-app уведомления

SSE Endpoint

```
// src/app/api/events/route.ts
export async function GET(request: Request) {
  const session = await auth();
  if (!session) return new Response("Unauthorized", { status: 401 });

  const stream = new ReadableStream({
    start(controller) {
      // Subscribe to Redis pub/sub for user-specific events
      const channel = `events:${session.user.id}`;
      // Push events to SSE stream
    },
    cancel() {
      // Unsubscribe from Redis
    },
  });

  return new Response(stream, {
    headers: {
      "Content-Type": "text/event-stream",
      "Cache-Control": "no-cache",
      "Connection": "keep-alive",
    },
  });
}
```

---

18. Security Architecture

Threat Model & Mitigations

Threat	Mitigation
XSS	DOMPurify for UGC (comments), CSP headers, React auto-escaping
SQL Injection	Prisma parameterized queries (never raw SQL with user input)
CSRF	NextAuth CSRF tokens, SameSite=Lax cookies
Sandbox escape	iframe sandbox attribute, CSP, no server-side code execution
API abuse	Redis rate limiting, per-endpoint limits
Payment tampering	HMAC verification, server-side amount validation
AI prompt injection	System prompt isolation, output sanitization
File upload attacks	Type validation (magic bytes), size limits, separate MinIO domain
Brute force auth	Rate limiting on auth endpoints, exponential backoff
Data exfiltration	RBAC on all endpoints, audit logging for admin actions

HTTP Security Headers

```
// next.config.ts headers
{
  "X-Frame-Options": "DENY",
  "X-Content-Type-Options": "nosniff",
  "Referrer-Policy": "strict-origin-when-cross-origin",
  "Permissions-Policy": "camera=(), microphone=(), geolocation=()",
  "Content-Security-Policy": "default-src 'self'; script-src 'self' 'unsafe-eval'
  "Strict-Transport-Security": "max-age=31536000; includeSubDomains"
}
```

Secrets Management

Все секреты хранятся в `.env` файле на сервере (никогда не в Git):

```
# .env (template)
DATABASE_URL=postgresql://vibelearn:***@postgres:5432/vibelearn
REDIS_URL=redis://redis:6379
MINIO_ENDPOINT=minio:9000
MINIO_ROOT_USER=***
MINIO_ROOT_PASSWORD=***
NEXTAUTH_SECRET=***
```

```
NEXTAUTH_URL=https://vibelearn.ru
GOOGLE_CLIENT_ID=***
GOOGLE_CLIENT_SECRET=***
OPENROUTER_API_KEY=***
CLOUDPAYMENTS_PUBLIC_ID=***
CLOUDPAYMENTS_API_SECRET=***
TELEGRAM_BOT_TOKEN=***
RESEND_API_KEY=***
SENTRY_DSN=***
```

---

19. Deployment & Infrastructure

Dokploy Configuration

```
Dokploy Project: vibelearn
├── Application: vibelearn-app
│   ├── Source: GitHub repo (main branch)
│   ├── Build: Dockerfile
│   ├── Domain: vibelearn.ru (Traefik auto-SSL)
│   ├── Port: 3000
│   ├── Health check: /api/health
│   └── Environment: .env variables
│
├── Application: vibelearn-bot
│   ├── Source: GitHub repo (main branch, context: ./bot)
│   ├── Build: bot/Dockerfile
│   ├── Port: 3001 (internal only)
│   └── Environment: .env variables
│
├── Database: PostgreSQL 16
│   ├── Volume: persistent
│   └── Backup: cron daily
│
├── Service: Redis (Valkey 8)
│   └── Volume: persistent
│
└── Service: MinIO
    ├── Volume: persistent
    ├── Console: minio.vibelearn.ru:9001
    └── API: s3.vibelearn.ru:9000
```

CI/CD Pipeline

```
Push to main
    │
    ▼
GitHub Actions / Dokploy auto-deploy
    │
    ├── 1. Run TypeScript checks (tsc --noEmit)
    ├── 2. Run linting (eslint)
    ├── 3. Run tests (vitest)
    ├── 4. Build Docker image
    ├── 5. Push to Dokploy
    ├── 6. Run Prisma migrations (prisma migrate deploy)
    └── 7. Health check → rollback if failed
```

DNS & Domains

```
vibelearn.ru        → Traefik → Next.js (port 3000)
s3.vibelearn.ru     → Traefik → MinIO API (port 9000)
minio.vibelearn.ru  → Traefik → MinIO Console (port 9001)
```

---

20. Observability

Logging (Pino)

Structured JSON logging to stdout, collected by Docker:

```
// src/lib/logger.ts
import pino from "pino";

export const logger = pino({
  level: process.env.LOG_LEVEL || "info",
  formatters: {
    level: (label) => ({ level: label }),
  },
});

// Usage:
logger.info({ userId, lessonId, xpGained }, "Lesson completed");
logger.error({ err, transactionId }, "Payment webhook failed");
```

Error Tracking (Sentry)

```
// sentry.server.config.ts
Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: process.env.NODE_ENV,
  tracesSampleRate: 0.1,  // 10% of transactions
  profilesSampleRate: 0.1,
});
```

Health Check

```
// src/app/api/health/route.ts
export async function GET() {
  const checks = {
    database: await checkPostgres(),
    redis: await checkRedis(),
    minio: await checkMinIO(),
    timestamp: new Date().toISOString(),
  };

  const healthy = Object.values(checks).every(v =>
    typeof v === "string" || v === true
  );

  return Response.json(checks, { status: healthy ? 200 : 503 });
}
```

Key Metrics to Track

- **Application:** request latency (p50, p95, p99), error rate, active sessions
- **AI:** tokens consumed, cost per day, latency per model, cache hit rate
- **Business:** DAU, enrollment rate, completion rate, revenue
- **Infrastructure:** CPU, RAM, disk usage per container

---

21. Performance Budget

Metric	Target	Measurement
TTFB (static pages)	< 200ms	Lighthouse
TTFB (dynamic pages)	< 500ms	Lighthouse

LCP (lesson page)	< 2.5s	Core Web Vitals
FID / INP	< 200ms	Core Web Vitals
CLS	< 0.1	Core Web Vitals
API response (95th)	< 500ms	Server metrics
AI first token	< 1.5s	Server metrics
JS bundle (initial)	< 150 KB gzipped	Build analysis
Concurrent users	500+	Load testing

Optimization Strategies

- **ISR (Incremental Static Regeneration)** for course catalog and public pages
- **RSC (React Server Components)** — minimize client JS
- **Dynamic imports** for heavy components (Monaco, React Flow, Tiptap)
- **Image optimization** via Next.js `<Image>` + sharp
- **Redis caching** for hot paths (course data, progress)
- **Connection pooling** via Prisma (pool_size = 10)
- **Streaming SSR** for lesson pages with heavy content

---

22. Disaster Recovery

Backup Strategy

Data	Method	Frequency	Retention
PostgreSQL	`pg_dump` → MinIO backup bucket	Daily 3:00 AM	30 days
MinIO media	Cross-bucket replication (or rsync to external)	Daily	30 days
Redis	RDB snapshots (built-in)	Every 60s	Ephemeral (cache only)

Environment	Encrypted .env in separate storage	On change	Permanent

Backup Script

```
#!/bin/bash
# /opt/vibelearn/backup.sh (cron: 0 3 * * *)
DATE=$(date +%Y-%m-%d)
BACKUP_FILE="vibelearn_${DATE}.sql.gz"

# Dump PostgreSQL
docker exec vibelearn-postgres pg_dump -U vibelearn vibelearn | gzip > /tmp/${BAC

# Upload to MinIO backup bucket
docker exec vibelearn-minio mc cp /tmp/${BACKUP_FILE} minio/backups/${BACKUP_FILE

# Cleanup: remove backups older than 30 days
docker exec vibelearn-minio mc rm --older-than 30d minio/backups/

# Optional: copy to external storage (S3, another server)
```

Recovery Procedure

1. **Database failure:** Restore from latest pg_dump backup
2. **Application failure:** Dokploy redeploy from Git (automatic rollback available)
3. **Full VPS failure:** Provision new VPS → Dokploy install → restore backups → update DNS
4. **Estimated RTO:** < 2 hours for full recovery

---

23. ADR Log

ADR-001: Monolith over Microservices

Decision: Single Next.js app + separate Telegram bot.
Context: Team of 1-2 developers, vibe-coding approach, need for speed.
Consequence: Simpler deployment, shared database, easier debugging. Can extract services later if needed.

ADR-002: PostgreSQL over Supabase

Decision: Self-hosted PostgreSQL on VPS instead of Supabase cloud.
Context: Full control over data, no vendor lock-in, VPS deployment requirement. Cost

optimization.
Consequence: Need to manage backups and updates ourselves. No built-in realtime (using SSE instead).

ADR-003: MinIO over Cloudflare R2

Decision: Self-hosted MinIO for object storage.
Context: VPS-first approach, no dependency on external storage. S3-compatible API allows easy migration to R2/S3 later.
Consequence: Disk space limited by VPS. CDN needs separate solution (Cloudflare/Bunny.net proxy).

ADR-004: grammY over Telegraf

Decision: grammY as Telegram bot framework.
Context: Better TypeScript support, active maintenance, conversation plugin for multi-step flows, smaller bundle.
Consequence: Slightly smaller community than Telegraf, but sufficient documentation.

ADR-005: Tiptap over BlockNote

Decision: Tiptap 2 for block editor.
Context: More mature, better extension system, compatible with ProseMirror ecosystem, better for custom node types.
Consequence: More setup required than BlockNote, but more flexible for custom content blocks.

ADR-006: SSE over WebSocket

Decision: Server-Sent Events for realtime features.
Context: AI streaming is inherently one-directional (server→client). Comments and notifications are low-frequency. SSE simpler to implement with Next.js Route Handlers.
Consequence: No bidirectional communication. If needed later (e.g., collaborative editing), can add WebSocket.

ADR-007: Telegram Bot as Separate Service

Decision: Bot runs as its own Docker container, not inside Next.js.
Context: Bot has different lifecycle (long-running process vs request/response), needs persistent connection to Telegram API. Separation allows independent restarts.
Consequence: Shared database access via same Prisma schema (npm workspace or copied schema). Need Docker networking between services.

This document is a living artifact. Update it as architectural decisions evolve.